The Quick Developers Guide to Migrate Their Apps to Android 11

Android 11 has been launched already and is soon to be released on Android phones from most of the major manufactures. Android 11 is much awaited with features like Enhanced 5G Support, Native Screen Recorder, Notifications Priority Conversations, Notification Chat Bubble, and many privacy enhancements like Scoped Storage and Package Visibility.

Most of the apps are already ready to rock Android 11, but for some of you who have still not acknowledged this change, it is high time and this article is specifically for you.

In this article, we will quickly start with the important stuff first and later will be reviewing some good-to-have enhancements and deprecations of existing APIs and features.

So without further ado, let's quickly review what Android 11 got for us.

Foreground Service Type

If your app targets Android 11 (API level 30) or higher and accesses the camera or microphone in a foreground service, then you will have to declare the `camera` or `microphone` foreground service types, respectively, as attributes of your `<service>` component.

```
<manifest>
    ...
    <service ...
        android:foregroundServiceType="location|camera|microphone" />
</manifest>
```

Add foreground service types of Work Manager Workers

Similar to foreground services, if you have a long-running worker that requires access to location, camera, or microphone, then you need to declare the worker's foreground service type in your app's manifest.

```
<service
 android:name="androidx.work.impl.foreground.SystemForegroundService"
    android:foregroundServiceType="location|camera|microphone"
    tools:node="merge" />
```

Note the Service name here.

```
android:name="androidx.work.impl.foreground.SystemForegroundService"
```

To learn more, refer to "Support for long-running workers".

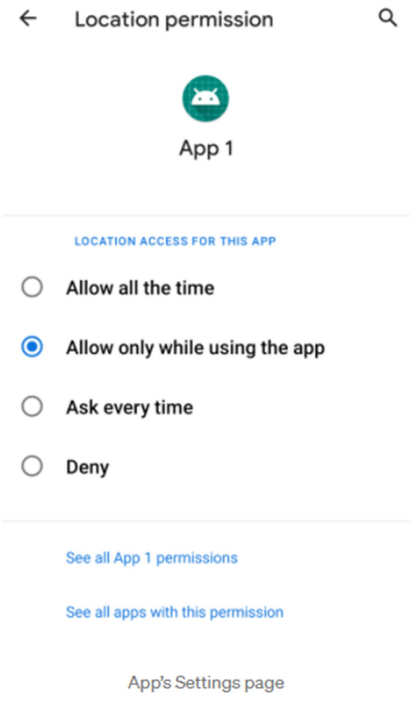
Background Location Permission

In the previous version of Android, ie. Android 10, we need to ask for Background Location permission in order for our app to access the location in the background.

In Android 11, it is taking one step further as now Background Permission cannot be given access before the Foreground Location Permission.

That means Foreground and Background Location Permissions cannot be requested together. If you request a foreground location permission and the background location permission at the same time, the system ignores the request and doesn't grant your app either permission.

Another important point to note is that beginning with Android 11, the system dialog doesn't include the **Allow all the time** option. Instead, users must enable the background location from the app's settings page.

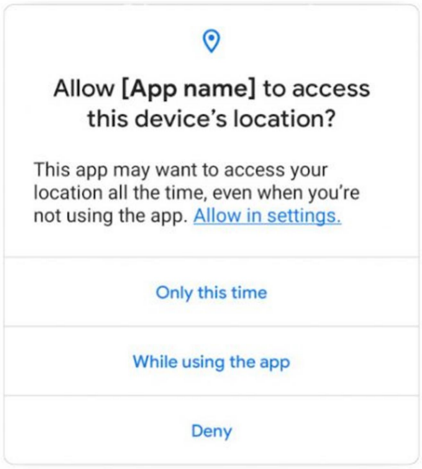


App's Settings page

Other enhancements in Permissions

One-Time Permissions

Considering the User's Privacy in mind, whenever your app requests permission related to location, microphone, or camera, Android now gives the option to the user to allow permission only once. This means as soon as your app gets killed, all the One-Time permissions will be revoked.



Permission Dialog in Android 11, with Background Location Permission request

Permission Dialog in Android 11, with Background Location Permission request

Some Important points here to keep in mind are:

- It is applicable only for Runtime permissions.

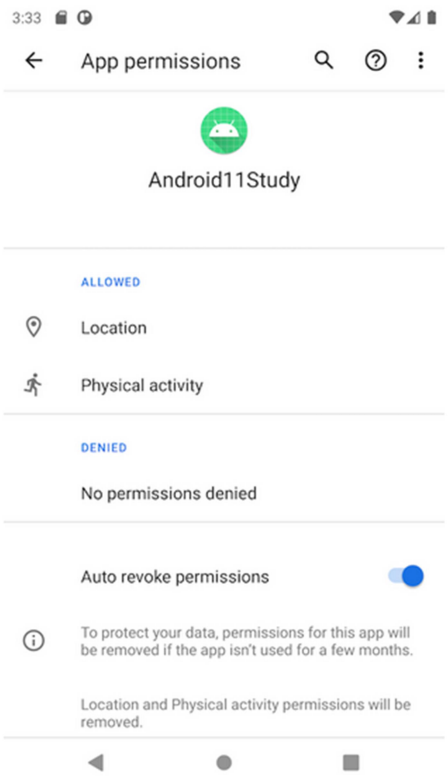
- If the user sends your app to the background, your app can continue to access the data for a short period of time.
- If you launch a foreground service while the activity is visible, and the user then moves your app to the background, your app can continue to access the data until that foreground service stops. And even if the user enters the activity again while the foreground service was running, then the granted permission will remain accessible.

Although for One-Time Permission you don't have to handle anything in your code, it is still worth mentioning, as it may affect your existing functionality especially if you are not following the best practices for requesting location permissions.

The system can Auto Revoke permissions from unused apps

If your app targets Android 11 or higher and isn't used for a few months, the system protects user data by automatically resetting the sensitive runtime permissions that the user had granted your app.

Auto Revoked Permissions can be disabled from the app's settings page, as shown below.



Alternatively, you can request the user to disable it by sending him to the permissions page, using action `Intent.ACTION_AUTO_REVOKE_PERMISSIONS`

~~*You can also configure Auto Revoke Permissions programmatically by using* `a-n-d-r-o-i-d-:-a-u-t-o-R-e-v-o-k-e-P-e-r-m-i-s-s-i-o-n-s-` *attribute in your apps Manifest file.*~~
Note that the `android:autoRevokePermission` *attribute is no longer used and will do nothing.*

Further, you can programmatically check if Auto Revoke Permissions are whitelisted or not

```
public boolean PackageManager#isAutoRevokeWhitelisted ()
```

```
private fun checkForAutoRevokePermission() {
        if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.R) {
            if (packageManager.isAutoRevokeWhitelisted) {
                // Do Work ....
                // ....
```

```
            } else {
                //Ask User to Disable Auto Revoke Permission from App's Settings Page
                startActivity(
                    Intent(
                        Intent.ACTION_AUTO_REVOKE_PERMISSIONS,
                        Uri.fromParts("package", packageName, null)
                    ).apply {
                        addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                    })
            }
        }
    }
```

Permission Dialog Visibility

Repeated denial of runtime permission by the user will imply “Don’t ask again”. The system then will no longer prompt the user with the permission dialog and you have to request the user to grant permission from the apps settings page.

Make sure you are already following the best practices to request runtime permissions.

Scoped Storage

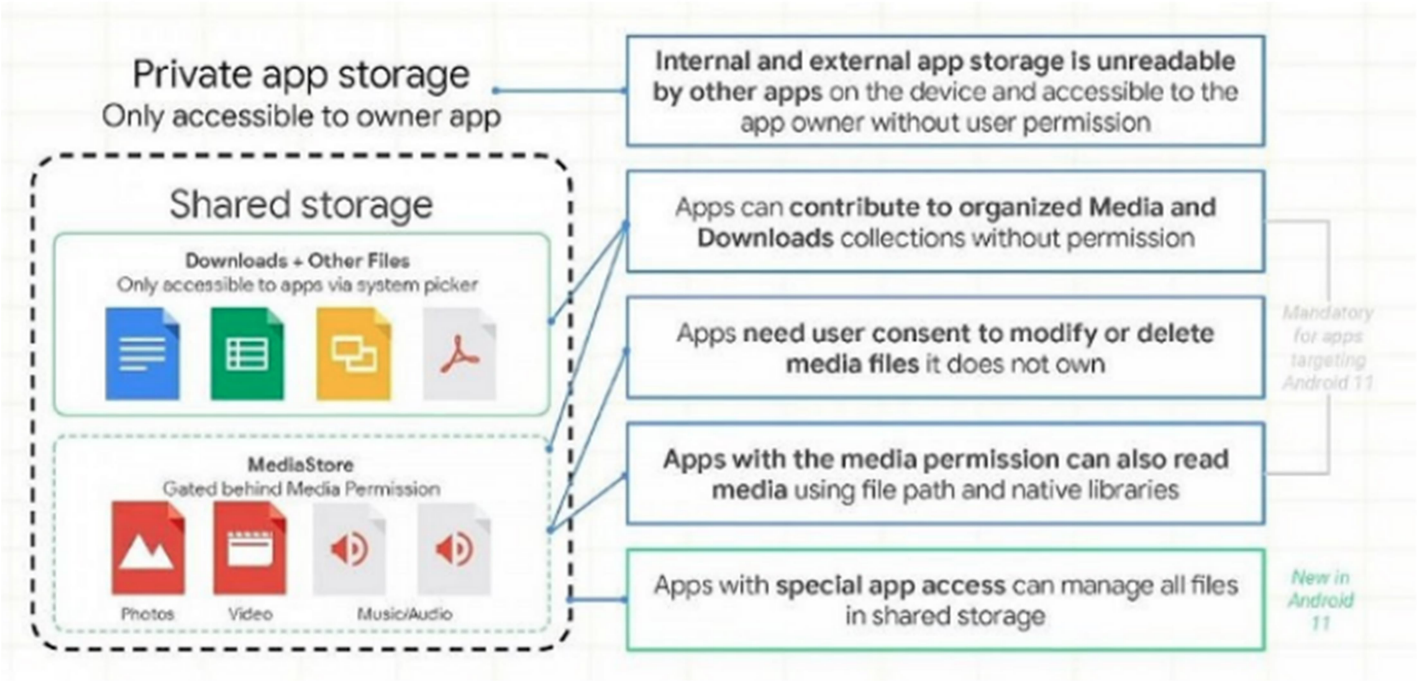
Scoped Storage was first introduced in Android 10, to give users more control over their files and to limit file clutter. What Scoped Storage really is, simply, instead of giving the app permission to view/modify whole external storage, give it access only to the app-specific directory on external storage, as well as specific types of media that the app has created.

Scoped Storage is necessary to —

- protect the user’s data
- prevent apps from scattering files all over the disc
- delete junk app files when that app is uninstalled

By introducing Scoped Storage, they separated storage into collections and limiting broad access to shared storage. Storage is divided into:

- Apps Private Storage — Internal or External storage accessible only by your app.
- Shared Storage — With is further categorized into Photos, Videos, Audio, and Downloads.



Source: Google

Source: Google

In Android 10 scoped storage was very controversial and thankfully optional.
But this is not the case with Android 11. With some changes to it, Scoped Storage is now mandatory for Android 11.

Luckily, in Android 11, apart from `MediaStore` APIs, we can use

- Regular Java's`File` API (with is internally delegated to `MediaStore` API)
- Native libraries, such as `fopen()`.

Modifying the media files is not as same as before, you require the user's consent for that. Using `createWriteRequest()` and `createDeleteRequest()` you can write and delete the media file respectively. Both methods support modifying multiple files (Bulk Operation) at the same time with a single request.

Also, two new Media Store concepts are introduced:

- Trash — Similar to a Recycle bin on Windows OS, where you can restore the media files or else they will be deleted permanently after some time. This can be achieved using`createTrashRequest()`.
- Favorites — Media files can be marked favorites by the user. This can be achieved using`createFavoriteRequest()`.

If your app still has files stored outside of the app and your app targets API level 29, use the *`requestLegacyExternalStorage`* attribute and migrate your files to the app directory.

Apps that run on Android 11 but target Android 10 (API level 29) can still request the *`requestLegacyExternalStorage`* attribute.

Refer to this helpful tutorial by Raywanderlitch: "Scoped Storage Tutorial for Android 11: Deep Dive" to help you get started with the Scoped Storage and alternatively you can refer to "Storage updates in Android 11" from the official Android Documentation.

Package Visibility

Earlier, apps could query the full list of installed apps on the system using methods like `queryIntentActivities()`. Mostly, I was far broader access you what you actually require.

With Android 11, this is no longer the case. Using the `<queries>` element, apps can define the set of other packages that they can access. This element helps encourage the principle of least privilege by telling the system which other packages to make visible to your app.

If your app targets Android 11 or higher, you might need to add the `<queries>` element in your app's manifest file. Within the `<queries>` element, you can specify packages by name, by intent signature, or by provider authority.

```
<manifest package="com.example.myapp">
  <queries>    <!-- Specific apps you interact with, eg: -->
    <package android:name="com.example.store" />
    <package android:name="com.example.maps" />

    <!--
         Specific intents you query for,
         eg: for a custom share UI
    -->
    <intent>
      <action android:name="android.intent.action.SEND" />
      <data android:mimeType="image/png" />
    </intent>  </queries>
  ...
</manifest>
```

Example

Suppose you want to send an email, you have to create an Intent for it and let the system find an appropriate app to handle the request.

```
val intent = Intent().apply {
    action = Intent.ACTION_SENDTO
    data = (Uri.parse("mailto:recipient@example.com"))
    putExtra(Intent.EXTRA_EMAIL, "Hi! How are you?")
    putExtra(Intent.EXTRA_SUBJECT, "Testing Package Visibility")
}startActivity(intent)
```

This code will work fine (even on the API level 30) if the email app is installed on the device. But what if there are no apps that can handle this request? — The app will crash by throwing `ActivityNotFoundException`.

A safe option is to first check that if we have at least one app which can handle this request. So we could do something like:

```
// It will always return null on Android 11, unless <queries> specified in manifest!if
(intent.resolveActivity(packageManager) == null) {
    toast("No app found to handle this request")
} else {
    startActivity(intent)
}
```

Now, this is where your code will **fail**, as your app is not allowed to query other installed apps. Here, in order for your code to work, you will have to add `<queries>` in your app's manifest file and declare your intent.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.android11study">

    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

```
    <uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
    <uses-permission android:name="android.permission.ACTIVITY_RECOGNITION" />


    <!-- Declare all the intents that your app will query -->
    <queries>
        <intent>
            <action android:name="android.intent.action.SENDTO" />
            <data android:scheme="mailto" />
        </intent>
    </queries>


    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Android11Study">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>

</manifest>
```

Custom Tabs

When using Custom Tabs to open a Web URL, it is possible that there may be some non-browser apps that can also handle that URL. It is recommended that you allow the non-browser app to handle that URL first.

`FLAG_ACTIVITY_REQUIRE_NON_BROWSER` can be used here to query Non-Browser apps only.

To learn more about it, refer to "Configuring package visibility based on use cases" from Android Documentation.

```
try {
    val url = "https://mylovelyapp.com/example"
    val intent = Intent(ACTION_VIEW, Uri.parse(url)).apply {
        // The URL should either launch directly in a non-browser app (if it's
        // the default), or in the disambiguation dialog.
        addCategory(CATEGORY_BROWSABLE)
        flags = FLAG_ACTIVITY_NEW_TASK or FLAG_ACTIVITY_REQUIRE_NON_BROWSER
    }
    startActivity(intent)
} catch (e: ActivityNotFoundException) {
    // Only browser apps are available, or a browser is the default.
    // So you can open the URL directly in your app, for example in a
    // Custom Tab.
    openInCustomTabs(url)
}
```

Important Point

The system makes some apps visible to your app automatically, but it hides other apps by default. Generally, these are apps that are linked to your app in some ways, like, Any app that starts or binds to a service in your app, access a content provider in your app, whose content provider your app accesses, etc.

You can start another app's activity using either an implicit or explicit intent, regardless of whether that other app is visible to your app. You just cannot query the intent.

What do I do if my App is an App Launcher or similar, where I need to know all the apps that are installed on the device?

To allow your app to see all other installed apps, the system provides the `QUERY_ALL_PACKAGES` permission.

```
<uses-permission android:name="android.permission.QUERY_ALL_PACKAGES"/>
```

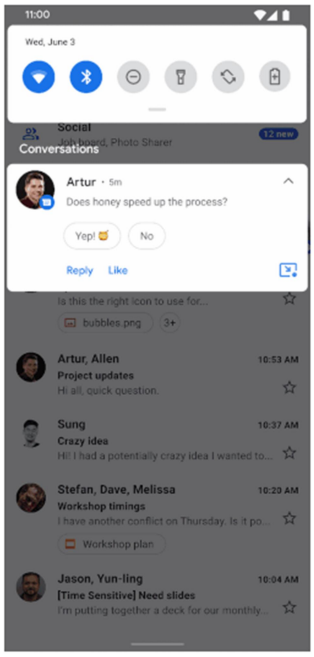
This permission is appropriate for apps that genuinely need this permission like a Launcher App, a Security App, Device Management App, Accessibility App, etc.

In an upcoming policy update, look for Google Play to provide guidelines for apps that need the `QUERY_ALL_PACKAGES` permission.

Good-to-Have Enhancements

Notifications

Notifications now have support for Priority conversations and native chat bubbles, and sending images right from the notification panel.

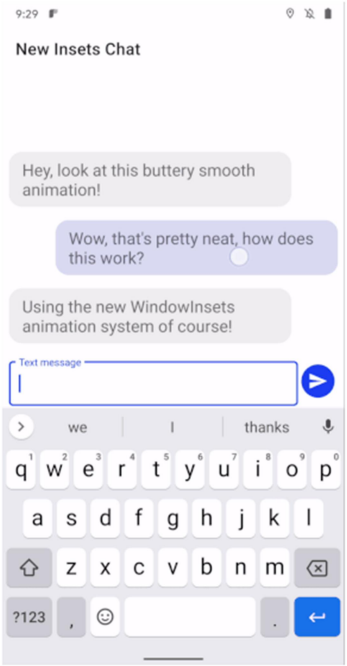


Conversations notifications are styled differently from other Non-Conversation notifications with a strong emphasis on the avatar representing people combined with the app carrying the conversation. These notifications bring some actions like marking this conversation as a priority, promoting this conversation as a bubble, setting custom sounds or vibrations, or simply silencing the notification.

To learn more about Conversation Space, `MessagingStyle`, and the Chat Bubble, refer to "People and conversations" from Android Documentation.

Synchronized IME transitions

New APIs let you synchronize your app's UI with the IME (input method editor, or on-screen keyboard) and system bars as they animate on and offscreen, making it much easier to create natural, intuitive, and jank-free IME transitions.



WindowInsetsAnimation.Callback API notifies apps of per-frame changes to insets while the system bars or the IME animate.

And moreover, finally, a question that was being asked for decades "How to check if On-Screen Keyboard is visible" can now finally be answered.

Read more about it in this detailed, amazing article by

Chris Banes
.

Deprecations

Async Task is deprecated

Starting from API level 30 `AsyncTask` is deprecated. It means you can still use it in your code, but it is no longer a recommended approach and you should soon migrate to `java.util.concurrent` or `Kotlin concurrency utilities` `(Kotlin Coroutines)` instead.

Android is now Kotlin first and it is very strongly recommended to use Kotlin Coroutines for such tasks.

The Non-Looper constructor of Handler is deprecated

`Handler()` and `Handler(Handler.Callback callback)` are deprecated because they do not specify a Looper, which can cause bugs, for example, mistakingly specifying a Handler on a thread without an active looper will cause your app to crash.

Android recommends explicitly passing a Looper to the Handler using these constructors: `Handler(Looper looper)` and `Handler(Looper looper, Handler.Callback callback)`

```
val mHandler: Handler = Handler(Looper.getMainLooper())
```

Custom Toast Views are deprecated

ie. Toast#setView and Toast#getView methods are now deprecated. Apps can create a standard text toast or use a Snackbar when in the foreground.

Also in order to make the UI consistent, starting from API level 30, **custom toast messages will not be displayed while the app is in the background.**

Another important thing here is that although Toast#setMargin and Toast#setGravity are not deprecated, but these methods are a no-op when called on text toasts. However, they work fine with simple toasts.

A "Simple Toast" is a toast constructed from `Toast(context: Context)` whereas a "Text Toast" is a toast constructed from the static method `Toast(context: Context, text: CharSequence, duration: Int)`